HACKATHON REVIEW DOCUMENT

MerchantMind

Agentic Commerce Intelligence for Physical POS Merchants

Submitted for	Theme
Pine Labs Playground 2026	**Agentic / Autonomous Commerce**
Repository	Stack
github.com/jainvpranav/MerchantMind	**Go · Python · Next.js · Amazon Bedrock**

> ***"Turn every Pine Labs payment into an autonomous growth action for the merchant — zero new hardware, one 2-second tap."***

Executive Summary

MerchantMind is a fully functional agentic commerce system built on top of Pine Labs POS terminals. It addresses a gap that existing tools — including Pine Labs' own analytics dashboard — do not cover: autonomous action on payment data, not just passive reporting.

14 million physical merchants in India process payments through Pine Labs terminals every day. Every transaction carries signal — what was bought, by whom, at what frequency. Today that signal disappears. MerchantMind captures it with a single 2-second category tag after each payment and runs three autonomous AI agent loops that act on the merchant's behalf.

KEY The core differentiation is not analytics. It is agentic action. Pine Labs can tell a merchant that Grocery sales spiked this week. MerchantMind tells the supplier and drafts the restock order.

What Has Been Built

As of submission, the following components are fully implemented and tested against live data:

- **Go event pipeline: terminal-gateway → Apache Kafka → event-consumer → PostgreSQL**
- Python ML engine: FP-Growth market basket analysis + RFM customer segmentation
- Three Claude-powered agent loops running on Amazon Bedrock (Sonnet for background agents, Haiku for real-time offer)
- Next.js merchant dashboard with campaign approval flow
- Go campaign-dispatcher wired to Twilio WhatsApp sandbox
- Go offer-engine-proxy with Redis caching for sub-2-second real-time offers
- Comprehensive seed dataset: 2,700+ transactions across 50 customer archetypes

The only integration points not yet live: Pine Labs terminal SDK hooks (post-payment webhook + pre-payment callback). All backend infrastructure is ready to receive those two calls.

Problem & Opportunity

The Gap in the Market

Small and mid-size merchants in India process hundreds of transactions per day but have no practical way to act on customer behaviour data. Enterprise tools that solve this problem — Salesforce Marketing Cloud, Clevertap, MoEngage — cost ₹10–50 lakh per year and require dedicated marketing teams to operate. They are out of reach for 99% of the Pine Labs merchant base.

Merchants on Pine Labs	14M+ physical POS merchants across India
Annual cost of alternatives	₹10–50L/year (Salesforce, Clevertap, MoEngage)
SMBs that can afford them	Less than 1%
Data already captured	Every transaction through the Pine Labs terminal
Data currently used	None — it flows through and disappears

Why Pine Labs Is the Right Platform

Pine Labs sits at the intersection of merchants, customers, and payment rails. No other player has a terminal network of this scale with this level of transaction-level data. Competitors like Clevertap rely on app events and CRM integrations — they cannot get to physical POS data natively. MerchantMind is built on Pine Labs' existing data moat.

NOTE Pine Labs recently partnered with OpenAI to embed AI into merchant reports. That partnership produces reactive intelligence — summarising what happened. MerchantMind is proactive autonomous action — doing something about it. These are complementary, not competing.

Solution — Three Autonomous Agents

After every payment, the merchant taps one of six category buttons (Grocery, Pharma, Clothing, Food & Bev, Electronics, Other) on a post-payment screen. This single tap attaches a category signal to the transaction and feeds the data pipeline. Three agent loops then run in the background.

01 Restock Agent

Trigger	Category purchase velocity spikes more than 30% above the rolling 7-day average
Action	Calculates estimated days until stockout at current rate
Output	Drafts a plain-language restock alert and queues it in the merchant dashboard
Merchant	Reviews draft and taps Approve — one tap triggers the supplier notification
Model	Claude Sonnet on Amazon Bedrock

WHY Merchants don't track inventory velocity manually. By the time they notice a stockout, it has already cost them sales. This agent catches it 6-10 days in advance.

02 Recovery Agent

Trigger	A customer with 3+ previous visits crosses 10+ days without returning (RFM at-risk threshold)
Action	Identifies the customer's preferred category and historical shopping window
Output	Generates a personalised WhatsApp win-back message with a small incentive
Merchant	Reviews draft and taps Approve — message dispatches via WhatsApp Business API
Model	Claude Sonnet on Amazon Bedrock

WHY Generic WhatsApp blasts get 2-3% open rates. Personalised messages sent at the customer's habitual shopping time get 20-30%. The agent does the personalisation automatically.

03 Dynamic Offer Agent

Trigger	A returning customer (recognised by phone hash) initiates payment on the terminal
Action	Looks up purchase history in Redis cache (<1ms) or runs Claude Haiku inference (<2s total)
Output	Surfaces the single best cross-sell offer on the POS screen before the transaction closes
Merchant	No action required — fully autonomous
Model	Claude Haiku on Amazon Bedrock (chosen for speed — hard 2-second SLA)
Fallback	If the agent exceeds 3 seconds or fails, payment proceeds normally — offer is never shown. Transactions are never blocked.

WHY The moment of payment is the highest-intent moment in a merchant's day. The customer is already buying. A relevant cross-sell shown at this exact moment converts at 3-5x the rate of any post-purchase campaign.

System Architecture

The system is divided into four layers. Each layer has a single responsibility and communicates with the next through well-defined interfaces.

Layer	Responsibility	Technology
01 Capture	Post-payment category tag, customer hash, transaction event	Go (terminal-gateway) · Pine Labs Terminal SDK
02 Ingest	Event queuing, normalisation, durable storage, cache	Apache Kafka · Go event-consumer · PostgreSQL · Redis
03 Intelligence	Basket analysis, customer segmentation, agent loops	Python · FP-Growth · RFM · Claude on Amazon Bedrock
04 Act	Dashboard, campaign approval, WhatsApp dispatch	Next.js · Go campaign-dispatcher · Twilio / WhatsApp API

Service Inventory

Service	Language	Port	Role
terminal-gateway	Go	8080	Receives POS webhook, validates, publishes to Kafka
event-consumer	Go	—	Reads Kafka topic, writes normalised rows to PostgreSQL
merchant-api	Go	8081	REST API powering the dashboard — 8 endpoints
offer-engine	Go	8082	Real-time offer proxy with Redis cache; calls Python agent
campaign-dispatcher	Go	—	Polls approved campaigns every 10s, dispatches via Twilio
basket_analysis	Python	—	FP-Growth pipeline — runs as nightly cron
rfm segmentation	Python	—	RFM scoring — runs as nightly cron
recovery_agent	Python	—	Claude tool-use loop — triggered by RFM at-risk detection
restock_agent	Python	—	Claude tool-use loop — triggered by velocity spike detection
dynamic_offer_agent	Python	5001	Claude Haiku tool-use loop — Flask HTTP server, called synchronously
Next dashboard	Next.js	3000	Merchant-facing UI: summary, patterns, segments, campaign approval

Data Model

Five PostgreSQL tables store the full state of the system:

Table	Purpose	Key Fields
merchants	One row per merchant	id (UUID), name
transactions	Every POS event — the core fact table	merchant_id, amount, category, customer_hash, transacted_at
basket_patterns	FP-Growth output — category co-occurrence rules	antecedent, consequent, confidence, support, computed_at
customer_segments	RFM output — one row per customer per merchant	customer_hash, segment, rfm_score, last_seen, avg_basket, visit_count
campaigns	Agent-generated drafts and their approval / send status	agent_type, status (draft/approved/sent/failed), message_body, scheduled_at, sent_at

Repository Structure

The repository is at github.com/jainvpranav/MerchantMind. It is organised into five top-level directories:

backend/	Five Go microservices: terminal-gateway, event-consumer, merchant-api, offer-engine, campaign-dispatcher. Each is an independent Go module with its own go.mod.
ml/	Python ML pipeline: basket_analysis/, segmentation/, agents/ (all three Bedrock agent files + shared bedrock_client.py), and scripts/ (seed_data.py, setup_and_run.py).
frontend/	Next.js 15 dashboard with Tailwind CSS, shadcn/ui, and Recharts. Four pages: Overview, Basket Patterns, Customers, Agent Actions.
infra/	docker-compose.yml spinning up PostgreSQL, Redis, Zookeeper, and Kafka locally with a single command.
scripts/	schema.sql for database initialisation, and the setup_and_run.py pipeline orchestrator.

Key Files for Review

File	Purpose
backend/terminal-gateway/main.go	Entry point — Gin POST handler, Kafka writer, event validation
backend/event-consumer/main.go	Kafka consumer with idempotent Postgres insert, transient/permanent error classification
backend/merchant-api/main.go	8 REST endpoints, CORS, structured query responses
backend/offer-engine/main.go	Redis cache layer + HTTP proxy to Python agent with 3s timeout and graceful fallback
backend/campaign-dispatcher/main.go	10-second poll loop, Twilio WhatsApp dispatch, demo phone number mode
ml/agents/bedrock_client.py	Shared Bedrock client — converts Anthropic SDK tool format to Bedrock converse API format
ml/agents/recovery_agent_bedrock.py	Recovery agent tool-use loop with get_at_risk_customers and draft_winback_campaign tools
ml/agents/restock_agent_bedrock.py	Restock agent with get_velocity_data, get_restock_history, draft_restock_alert tools
ml/agents/dynamic_offer_agent_bedrock.py	Real-time offer agent — Flask HTTP server mode for synchronous POS integration

File	Purpose
ml/scripts/seed_data.py	50 customer archetypes across 8 behavioural profiles with baked-in basket patterns and velocity spikes
ml/scripts/setup_and_run.py	One-command pipeline runner: infra check → seed → basket analysis → RFM → agents
infra/docker-compose.yml	Local infrastructure: Postgres 15, Redis 7, Confluent Kafka 7.5

AI Agent Design

Why Tool-Use Agents vs. Simple LLM Calls

Each agent is implemented as a Claude tool-use loop rather than a single prompt-response call. This was a deliberate architectural choice. A single prompt cannot verify that a campaign was actually saved, cannot retrieve live customer data to reason about, and cannot chain multiple dependent actions in a single flow.

The tool-use pattern allows each agent to:

- Retrieve live data from Postgres before deciding whether to act
- Chain dependent actions (e.g. check velocity → check history → draft alert, only if threshold is met)
- Write its output back to the database as a structured artefact rather than as free text
- Be inspectable — every tool call and its inputs are logged

Bedrock vs. Anthropic SDK

The agents use Amazon Bedrock rather than the Anthropic API directly. The shared bedrock_client.py helper handles the format differences invisibly:

Difference	Anthropic SDK	Bedrock Converse API
Tool format	input_schema (flat)	toolSpec.inputSchema.json (wrapped)
Response field	tool_use / id	toolUse / toolUseId
Model IDs	claude-sonnet-4-20250514	us.anthropic.claude-sonnet-4-5-20250929-v1:0
Auth	ANTHROPIC_API_KEY env var	IAM role (EC2) or AWS_PROFILE (local)
Region routing	Global	us. prefix = cross-region inference profile

SECURITY The system uses IAM roles for Bedrock authentication — no long-lived access keys in code or environment variables. On EC2, boto3 retrieves short-lived tokens from the instance metadata service automatically. Locally, a named AWS profile in ~/.aws/credentials is used.

Demo Flow

The demo can be run end-to-end in approximately 4 minutes. The recommended sequence for a live presentation:

Step	Action	What the Audience Sees
1	Navigate to /terminal in the dashboard	Mock POS terminal screen — amount input, Pay button
2	Enter ₹450 and tap Pay	Payment processing screen, then Dynamic Offer Agent surfaces an offer in <2 seconds
3	Tap Grocery on the category tag screen	Transaction fires to terminal-gateway, Kafka event visible in terminal logs
4	Open merchant dashboard overview	Revenue cards, category velocity chart showing Grocery spike, customer segment donut
5	Navigate to Basket Patterns	FP-Growth rules: Grocery → Dairy (67%), Grocery → Food & Bev (55%), Pharma → Personal Care (40%)
6	Navigate to Agent Actions — Restock tab	Draft restock alert for Grocery with estimated 6-day stockout
7	Tap Approve on the restock alert	Status updates to Sent, campaign-dispatcher fires
8	Navigate to Agent Actions — Recovery tab	Draft win-back WhatsApp for at-risk customer C3004 with personalised message
9	Tap Approve	WhatsApp message arrives on demo phone live on screen within 5 seconds
10	Close: show the full pipeline in one view	"Pine Labs already has the data. MerchantMind makes it act."

Pre-Demo Checklist

- docker ps — verify all 4 containers running (postgres, redis, zookeeper, kafka)
- go run main.go in terminal-gateway, event-consumer, merchant-api, offer-engine, campaign-dispatcher
- python dynamic_offer_agent_bedrock.py --server running on port 5001
- npm run dev in frontend/ — dashboard live at localhost:3000
- DEMO_PHONE_NUMBER set in campaign-dispatcher .env — your phone pre-registered in Twilio sandbox
- Run setup_and_run.py --skip-agents first to ensure seed data and ML tables are populated

Business Model

MerchantMind is designed as a revenue layer sitting on top of Pine Labs' existing merchant relationships. Three distinct revenue streams, each independently viable:

Stream	Model	Price Point	Who Pays
SaaS Analytics Tier	Monthly subscription	₹500–2,000/merchant/month	Merchant
Campaign Performance Fee	% of attributed GMV recovered	2–5% of recovered spend	Merchant (pay on success)
Offer Placement Fee	CPC / CPM at point of payment	Negotiated per brand	Brands / large distributors

NETWORK EFFECT

Every new merchant adds transaction data that improves basket analysis accuracy across the network. Cross-merchant patterns (e.g. customers who buy Grocery at Store A also buy Pharma at Store B) create a defensible moat that no standalone analytics vendor can replicate.

Integration Requirements

Two integration points with Pine Labs are required to take this from a working POC to a fully live system. Everything else is already built and tested.

01 Terminal Webhook — Post-Payment

After a successful payment on a Pine Labs Plutus terminal, fire a POST request to:

```
POST http://merchantmind-server/v1/transaction/tag
{
  "merchant_id":   "<uuid>",       // Pine Labs merchant identifier
  "amount":        450.00,          // Transaction amount
  "category":      "Grocery",       // Merchant-tapped category
  "customer_hash": "1234"           // Optional: hashed last-4 phone
digits
}
```

Impact on payment flow: zero. This call happens after the payment success screen. The terminal does not wait for a response. Fire-and-forget.

02 Pre-Payment Callback — Dynamic Offer

Before displaying the payment confirmation screen, call:

```
POST http://merchantmind-server/v1/offer/realtime
{ "merchant_id": "<uuid>", "customer_hash": "1234" }
// Response (< 2 seconds):
{ "has_offer": true,  "display_text": "Add dairy — ₹30 cashback on ₹200+",
"elapsed_ms": 1240 }
{ "has_offer": false, "reason": "No matching offer for this customer" }
```

If has_offer is true, display display_text on the terminal screen. If has_offer is false, or if the request times out, proceed normally — payment is never blocked.

Risks & Mitigations

Risk	Likelihood	Mitigation
Merchant taps category lazily (all transactions tagged as Grocery)	Medium	Per-merchant data quality score flags uniform distributions. Low-quality merchants are excluded from ML runs until data improves.
Dynamic Offer Agent exceeds 2-second SLA	Medium	Redis cache serves most returning customers in <1ms. Claude Haiku (not Sonnet) used for speed. Hard 3s timeout in Go offer-engine — returns no-offer fallback, never blocks payment.
Claude API rate limit or outage during demo	Low	Background agents (Restock, Recovery) pre-run before demo. Dynamic offer has rule-based fallback at 3s. Demo seed data ensures campaigns already exist in draft state.
No customer identity without phone hash	Medium	Basket analysis works at session level without identity. Dynamic Offer and Recovery agents degrade gracefully — no offer / no campaign rather than wrong offer / wrong campaign.
Campaign over-messaging (spam)	Low	Hard frequency cap: max 2 messages per customer per week enforced at campaign-dispatcher layer regardless of how many campaign drafts are approved.

Next Steps

Immediate (Hackathon → Demo)

- Integrate Pine Labs terminal SDK — wire the two webhook calls described in Section 9
- Production WhatsApp Business API approval (currently using Twilio sandbox)
- Deploy backend services to AWS EC2 (ap-south-1) with RDS PostgreSQL and ElastiCache Redis
- Deploy dashboard to Vercel — public URL for demo

Post-Hackathon (4 Weeks)

- Multi-merchant support: PostgreSQL Row Level Security + per-merchant data isolation
- Offer management UI: merchant-facing screen to create and manage the offer pool for Agent 3
- Airflow-scheduled ML pipeline: replace manual cron with DAG-based basket analysis + RFM runs
- Agent 3 cache warm-up: pre-compute offers for known returning customers at store opening time

Pilot (7 Weeks)

- 10-merchant pilot in one city (Bengaluru or Mumbai recommended — high Pine Labs density)
- A/B test: 5 merchants with MerchantMind vs 5 control merchants — measure recovery rate and basket size
- SaaS pricing rollout: free tier (dashboard only) + paid tier (agents active)

CLOSING The infrastructure is ready. The agents are running. The data pipeline is live. We need the terminal. Two webhook calls and MerchantMind is fully operational on every Pine Labs POS in India.

MerchantMind · Pine Labs Playground 2026

github.com/jainvpranav/MerchantMind

Agentic / Autonomous Commerce & Intelligent Payments